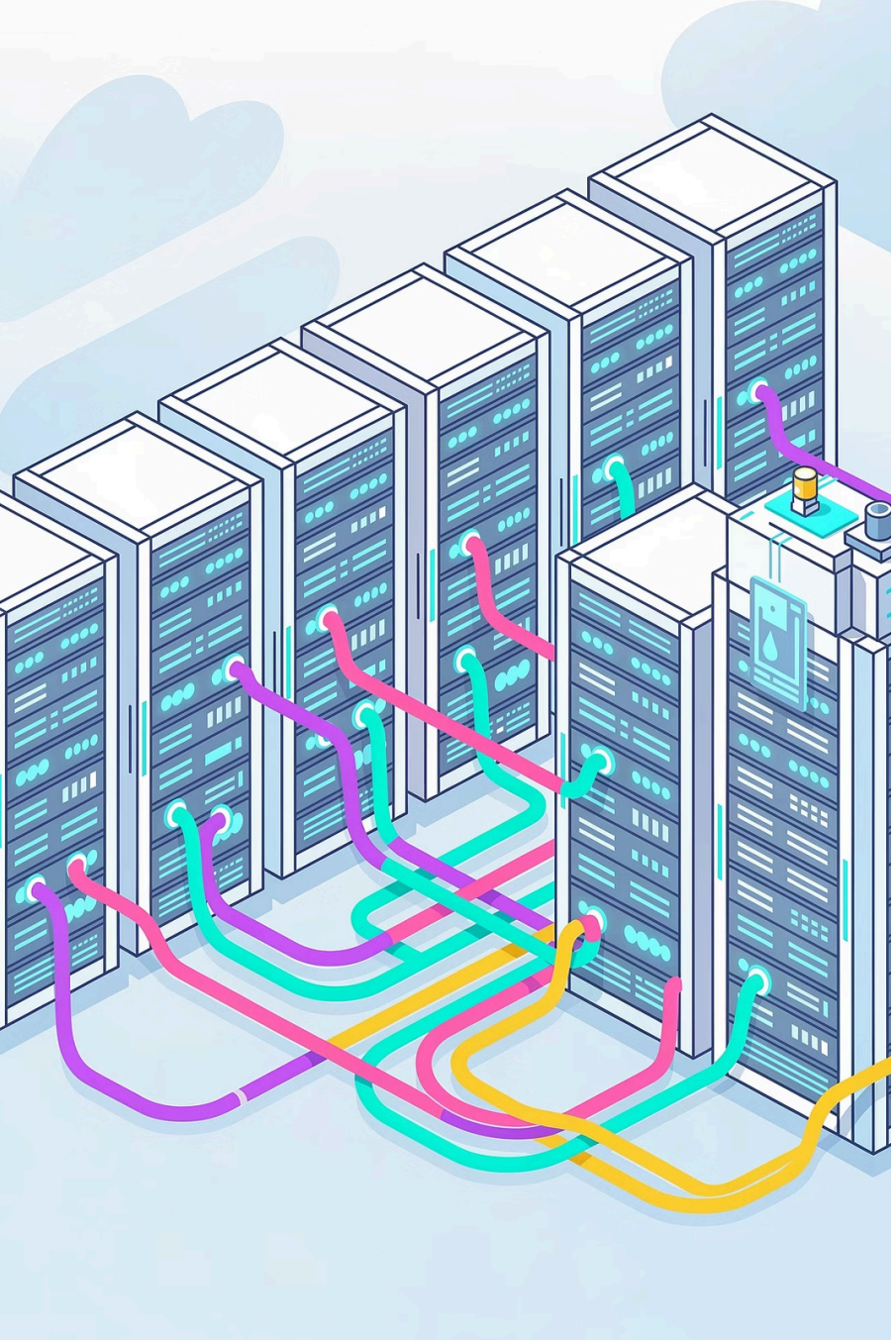


Introdução ao Ecossistema Hadoop

BIG DATA DIDÁTICO

Bem-vindo a uma jornada completa pelo universo do Hadoop! Nesta aula, vamos desvendar os fundamentos do processamento distribuído de dados em larga escala. Você vai entender como o Hadoop revolucionou o tratamento de Big Data e por que ele continua relevante no ecossistema moderno de ciência de dados. Prepare-se para explorar os componentes essenciais, compreender suas interações e identificar os pontos críticos que todo profissional precisa conhecer.:

github.com/jbruno97/ecossistema-hadoop

Objetivos da Aula

01

Fundamentos de Data Science

Compreender a disciplina no contexto de Big Data e suas aplicações práticas

02

Origem do Hadoop

Entender por que o Hadoop foi criado e qual problema ele resolve

03

Arquitetura e Componentes

Conhecer os componentes principais do ecossistema e suas funções

04

Limitações e Boas Práticas

Identificar pontos de atenção, desafios e como superá-los

05

Ambiente Prático

Visualizar como montar um ambiente de teste usando Docker ou VM

O que é Data Science?

Data Science é uma disciplina interdisciplinar que combina estatística, programação, conhecimento de negócio e machine learning para extrair insights valiosos de dados. É o encontro entre a matemática rigorosa e a aplicação prática em problemas reais.

Coleta

Obtenção de dados brutos

Limpeza

Tratamento e preparação

Análise

Exploração e descoberta

Modelagem

Machine Learning aplicado

Visualização

Comunicação de resultados

Decisão

Insights acionáveis

Exemplos práticos estão por toda parte: sistemas de recomendação da Netflix que sugerem o próximo filme perfeito, algoritmos de detecção de fraudes bancárias que protegem milhões de transações, e análises genômicas que aceleram descobertas médicas.

Big Data e os 5 Vs

Volume

Terabytes a petabytes de dados armazenados e processados

Velocidade

Geração contínua e necessidade de processamento em tempo real

Variedade

Estruturado, semi-estruturado e não-estruturado

Veracidade

Qualidade, confiabilidade e incerteza dos dados coletados

Valor

Potencial real de transformar dados em insights úteis para o negócio

Quando Big Data Encontra Data Science

Ferramentas tradicionais como Excel ou bibliotecas Python rodando em uma única máquina simplesmente não conseguem lidar com datasets massivos. Imagine tentar processar 100 terabytes de logs de sensores IoT ou analisar o comportamento de 500 milhões de usuários em uma planilha — impossível, certo?

O Problema

Datasets muito grandes ultrapassam a memória e capacidade de processamento de máquinas individuais, tornando a análise inviável com ferramentas convencionais.

A Solução

Processamento paralelo e distribuído permite dividir o trabalho entre dezenas ou centenas de máquinas, processando dados onde eles estão armazenados.

Casos de Uso

Análise preditiva em escala, feature engineering massivo para modelos de ML, processamento de logs em tempo real, e agregações complexas sobre bilhões de registros.

É aqui que o Hadoop entra como uma solução open-source pioneira, especialmente projetada para processamento batch de Big Data. Ele democratizou o acesso a tecnologias que antes eram exclusivas de gigantes como Google e Facebook.

Origem e Filosofia do Hadoop

História

O Hadoop nasceu da necessidade de indexar toda a web. Em 2003-2004, o Google publicou papers revolucionários sobre o Google File System (GFS) e o paradigma MapReduce. Inspirados por essas ideias, Doug Cutting e Mike Cafarella iniciaram o projeto Apache Hadoop em 2006, originalmente parte do projeto Nutch de busca web.

O nome "Hadoop" vem do elefante de pelúcia amarelo do filho de Doug Cutting — um nome simples para uma tecnologia que transformaria o mundo dos dados.

Commodity Hardware

Usa máquinas baratas e comuns em vez de supercomputadores caros

Escalabilidade Horizontal

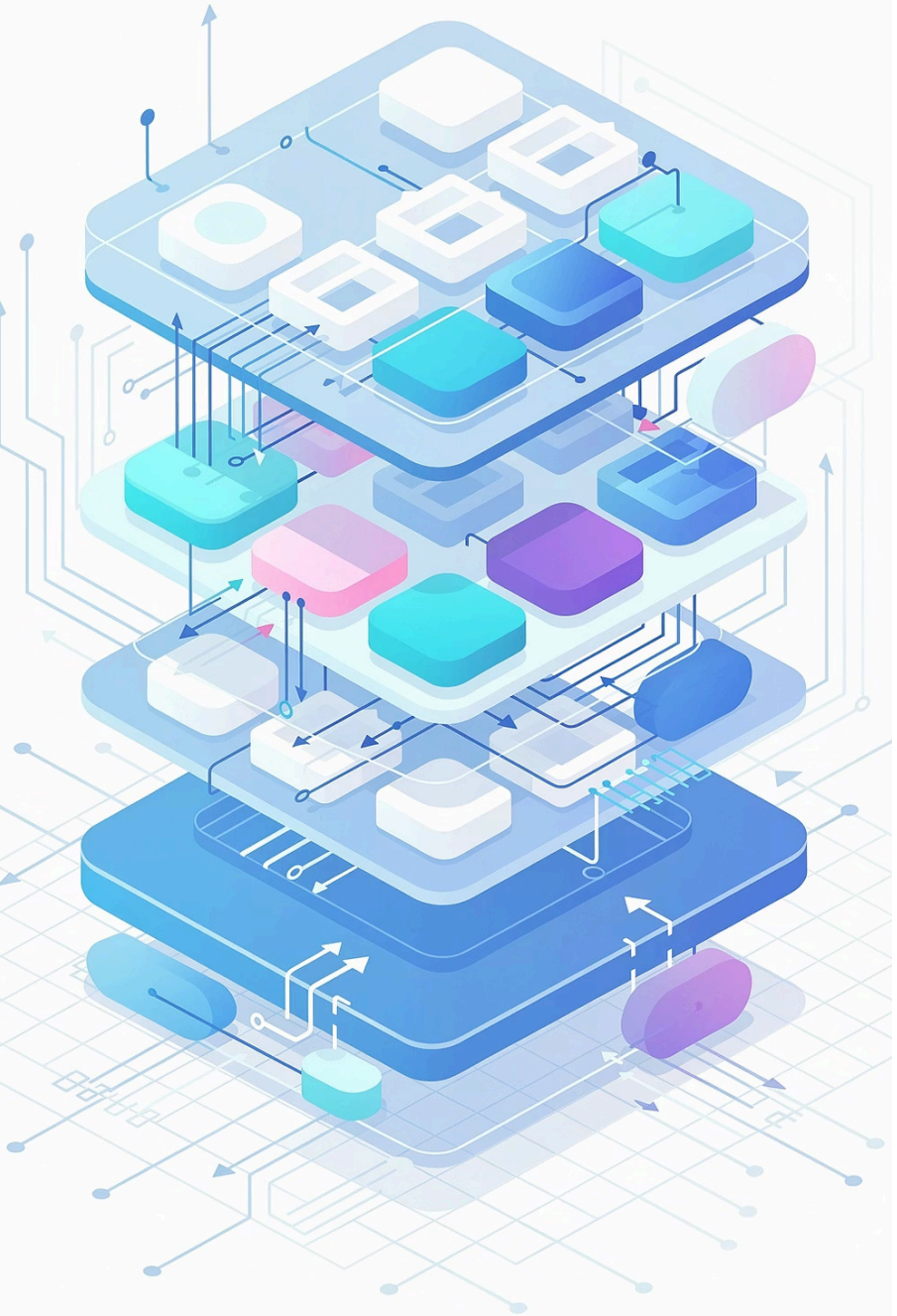
Cresce adicionando mais nós ao cluster

Tolerância a Falhas

Replicação automática de dados garante disponibilidade

Data Locality

Processa dados onde estão armazenados, reduzindo transferências



Visão Geral do Ecossistema Hadoop

Hadoop não é um software único, mas sim um ecossistema modular de ferramentas que trabalham juntas. Cada componente resolve um problema específico, e a beleza está na integração entre eles.

Camada de Armazenamento

HDFS para dados distribuídos, HBase para acesso NoSQL columnar em tempo real

Camada de Processamento

MapReduce para batch jobs, YARN como gerenciador de recursos do cluster

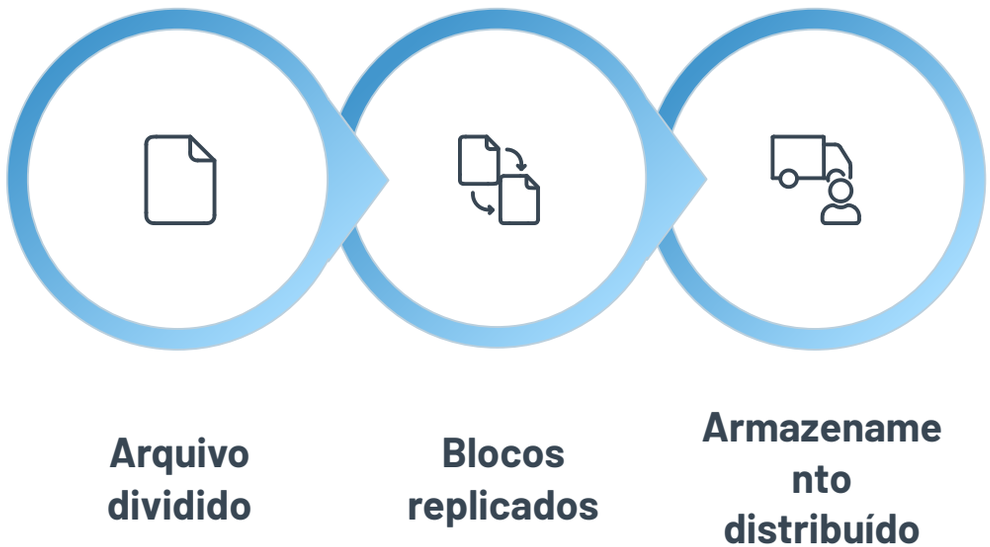
Camada de Acesso

Hive traz SQL para analistas, Pig oferece scripting procedural, Sqoop conecta bancos relacionais

Camada de Gerenciamento

Oozie orquestra workflows complexos, Zookeeper coordena serviços distribuídos, Flume/Kafka capturam streams

HDFS – Conceito e Propósito



O HDFS é a espinha dorsal do armazenamento Hadoop, projetado para cenários específicos de Big Data.

Hadoop Distributed File System

O HDFS é um sistema de arquivos distribuído e tolerante a falhas, otimizado para throughput alto em vez de latência baixa. Ele foi projetado com premissas claras: arquivos muito grandes (gigabytes a petabytes), padrão write-once/read-many, e hardware que pode falhar a qualquer momento.

- **Blocos grandes:** 128 MB por padrão (Hadoop 2+), otimizando para leituras sequenciais massivas
- **Replicação tripla:** Fator 3 por padrão garante que dados sobrevivam a falhas de até 2 nós
- **Ideal para Data Science:** Perfeito para armazenar datasets brutos, logs de sensores, dados históricos

O HDFS não é adequado para arquivos pequenos ou para aplicações que precisam de edições randômicas frequentes.

Arquitetura do HDFS

O HDFS segue uma arquitetura master-slave clássica, com papéis bem definidos para cada componente:

NameNode (Mestre)

Gerencia todos os metadados do sistema: namespace de diretórios, localização de blocos, permissões. É o cérebro do HDFS, mas não armazena dados reais.

DataNodes (Escravos)

Armazenam os blocos reais de dados. Enviam heartbeats periódicos ao NameNode para confirmar que estão ativos e reportam quais blocos possuem.

Secondary NameNode

Realiza checkpoints periódicos dos metadados do NameNode. Atenção: não é um backup nem um failover automático!

Fluxo de leitura: Cliente solicita arquivo → NameNode retorna localização dos blocos → Cliente lê diretamente dos DataNodes. **Fluxo de escrita:** Cliente pede ao NameNode para criar arquivo → NameNode aloca blocos → Cliente escreve pipeline nos DataNodes com replicação automática.

Pontos de Atenção no HDFS

Single Point of Failure

O NameNode é crítico — se ele falhar, todo o cluster fica inacessível. Solução: configurar HA (High Availability) com Zookeeper e NameNodes em standby.

Problema de Arquivos Pequenos

Milhões de arquivos pequenos sobrecarregam o NameNode com metadados. Cada arquivo consome ~150 bytes de memória. Solução: agrupar arquivos pequenos em SequenceFiles ou Parquet.

Limitações de Edição

HDFS é append-only — você não pode editar no meio de um arquivo. Isso é por design, otimizando para write-once/read-many workloads.

Segurança

Por padrão, HDFS tem segurança mínima. Em produção, é essencial configurar Kerberos para autenticação e encryption at rest (disponível no Hadoop 3+).

Para projetos de Data Science, o HDFS é ideal para armazenar raw data (logs, sensores, datasets brutos) que serão processados em batch, mas não para dados que precisam de acesso randômico frequente.

MapReduce – Paradigma de Processamento

MapReduce é um modelo de programação funcional que permite processar terabytes de dados de forma distribuída, mesmo sem experiência prévia em sistemas distribuídos. Sua elegância está na simplicidade: você escreve apenas duas funções.

1

Map

Processa dados localmente, transformando cada registro em pares (chave, valor)

2

Shuffle & Sort

Framework agrupa automaticamente todos os valores por chave e os redistribui

3

Reduce

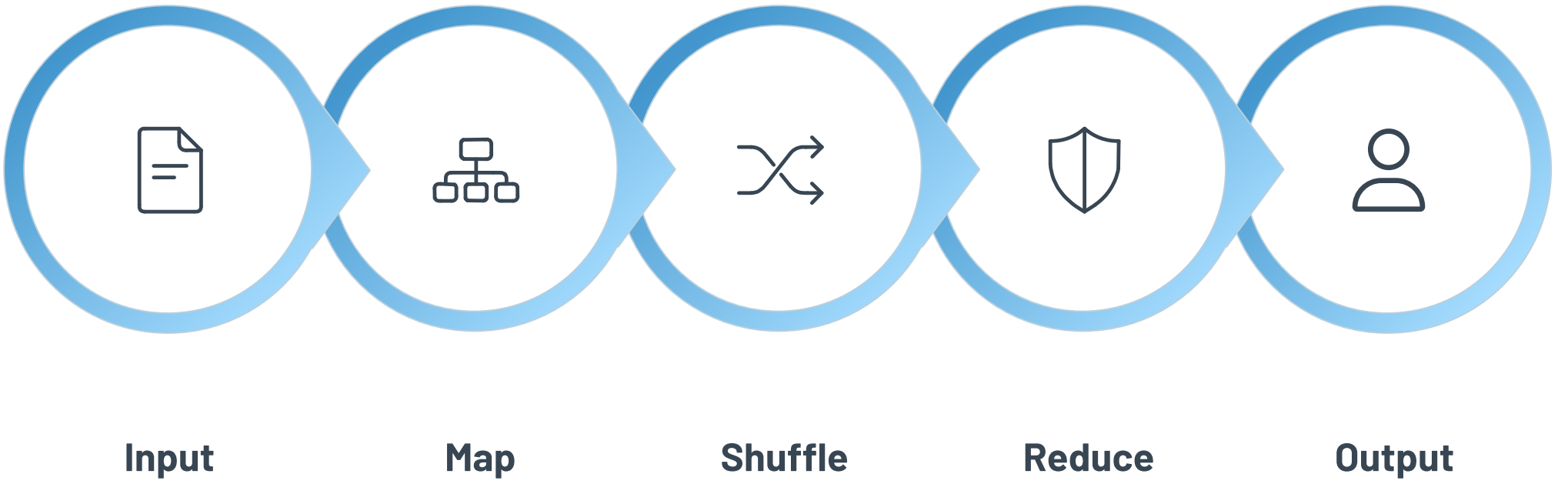
Agrega todos os valores de cada chave, produzindo o resultado final

Data Locality: A Magia

O MapReduce move o código para onde os dados estão, não o contrário. Isso minimiza a transferência de dados pela rede — o maior gargalo em sistemas distribuídos.

Exemplo clássico: Word Count conta palavras em bilhões de documentos. Map emite (palavra, 1) para cada ocorrência, Reduce soma todas as contagens por palavra.

Fluxo do MapReduce (Word Count)



Vamos detalhar cada etapa do processamento MapReduce usando o exemplo do Word Count:

1. **Input Split**
   Arquivo dividido em blocos de 128MB, um por Map task
2. **Map**
   Cada palavra vira (palavra, 1)
3. **Combiner**
   Mini-reduce local otimiza transferência de rede
4. **Shuffle**
   Dados movidos pela rede e agrupados por chave
5. **Reduce**
   Soma final por palavra
6. **Output**
   Resultado gravado no HDFS

Intermediate data pode ser gravado em disco se não couber na memória (spill), o que impacta performance. O Partitioner determina qual Reducer recebe qual chave.

Pontos de Atenção no MapReduce

Limitações Importantes

MapReduce tem overhead significativo: múltiplas gravações em disco, transferências de rede, e inicialização de processos. Para jobs iterativos (como treinamento de modelos de Machine Learning), isso se torna proibitivamente lento.

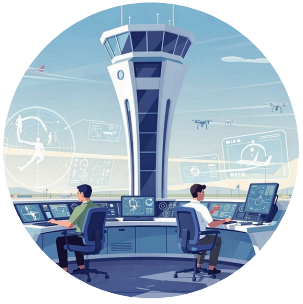
Foi exatamente essa limitação que motivou a criação do Apache Spark, que processa dados em memória e é 10-100x mais rápido para workloads iterativos.

- **Overhead alto:** Muitos passos de leitura/escrita em disco e transferências de rede entre fases
- **Lento para iterações:** Cada iteração de um algoritmo ML reinicia todo o pipeline, perdendo contexto
- **Debugging complexo:** Rastrear problemas em jobs distribuídos entre dezenas de máquinas é desafiador
- **Melhor caso de uso:** ETL batch massivo, processamento de logs históricos, agregações únicas sobre datasets enormes

Apesar das limitações, MapReduce ainda é útil em Data Science para ETL batch de datasets massivos onde a latência não é crítica.

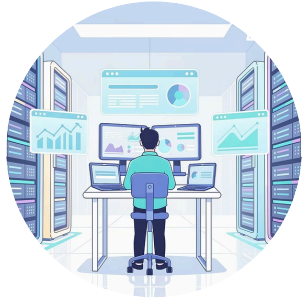
YARN – Gerenciamento de Recursos

YARN (Yet Another Resource Negotiator) foi introduzido no Hadoop 2.0 para separar o gerenciamento de recursos do processamento de jobs. Antes, o MapReduce cuidava de tudo, limitando o cluster a apenas esse framework. YARN permite rodar múltiplos frameworks simultaneamente: MapReduce, Spark, Flink, Tez, e outros.



ResourceManager

Mestre global que gerencia recursos de todo o cluster e arbitra demandas entre aplicações



NodeManager

Agente em cada nó que monitora recursos locais (CPU, memória) e executa containers

ApplicationMaster

Um por aplicação, negocia recursos com o ResourceManager e monitora execução de tarefas

Containers

Unidade de alocação que encapsula CPU e memória para executar tarefas isoladamente

Componentes Complementares e Evolução

Ferramentas do Ecossistema

Hive

SQL sobre HDFS, ideal para analistas que conhecem SQL

Pig

Scripting procedural para ETL complexo

HBase

NoSQL columnar para acesso randômico rápido

Sqoop

Import/export entre HDFS e bancos relacionais

Flume/Kafka

Ingestão de dados em streaming

Oozie

Orquestração de workflows complexos

Evolução e Contexto Moderno

É importante reconhecer que Hadoop é batch-oriented — lento para processamento real-time. Para essas necessidades, Spark Streaming ou Apache Flink são mais adequados.

Clusters Hadoop reais consomem muitos recursos. Em produção, segurança com Apache Ranger e Kerberos é essencial. O Hadoop 3.x trouxe melhorias importantes: Erasure Coding reduz overhead de storage, suporte a GPUs, e YARN aprimorado.

Integração moderna: Hoje, Hadoop funciona bem integrado com Spark e plataformas cloud como AWS EMR e Google Cloud Dataproc, que abstraem a complexidade de gerenciar clusters.

> Para iniciantes, recomendamos começar com ambientes Docker ou VMs locais (Cloudera Quickstart, Hortonworks Sandbox) antes de partir para clusters em produção.